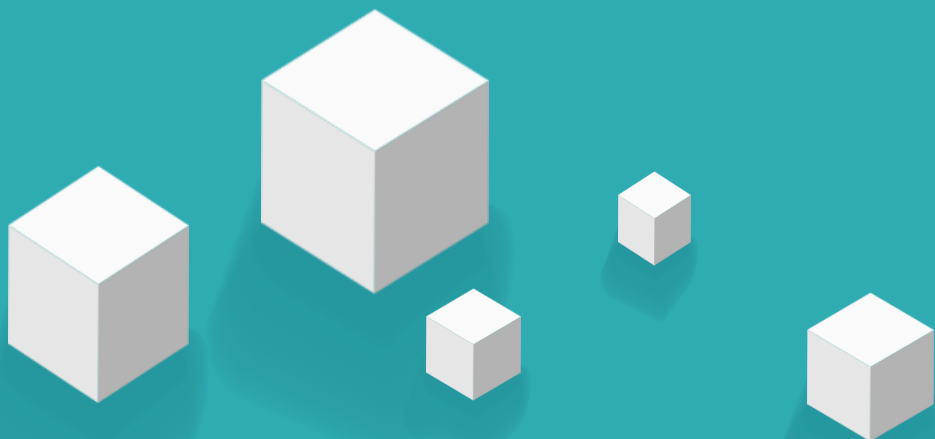
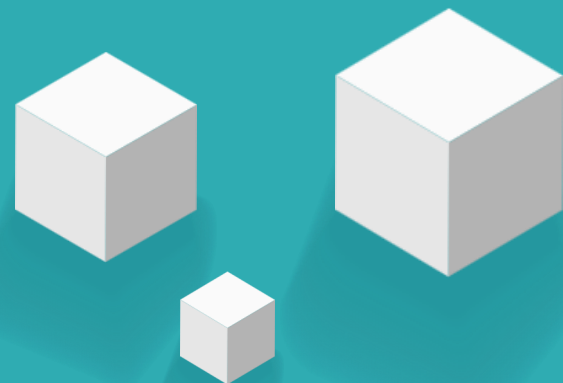


vue.js
Aug 12

第六章 样式绑定



» 第6章 样式绑定

- 绑定class属性
- 绑定style属性

» 6.1 绑定class属性

- Vue中通过更改元素的class属性可以更改元素的样式，而对class属性的操作可以通过v-bind来实现。
- 使用v-bind绑定class属性时，可以将class属性值绑定为字符串、对象或数组。

»» 6.1 绑定class属性

1. 将class属性值绑定为字符串

在Vue中，可以将class属性值绑定为字符串。

```
<template>
  <div v-bind:class="className"> </div>
                                类名
</template>
<script setup>
const className = 'box'
</script>
```

当上述代码运行后，div元素的渲染结果如下。

```
<div class="box"> </div>
```

» 6.1 绑定class属性

——为class属性绑定字符串

- 步骤1：创建src\components\ClassStr.vue文件。

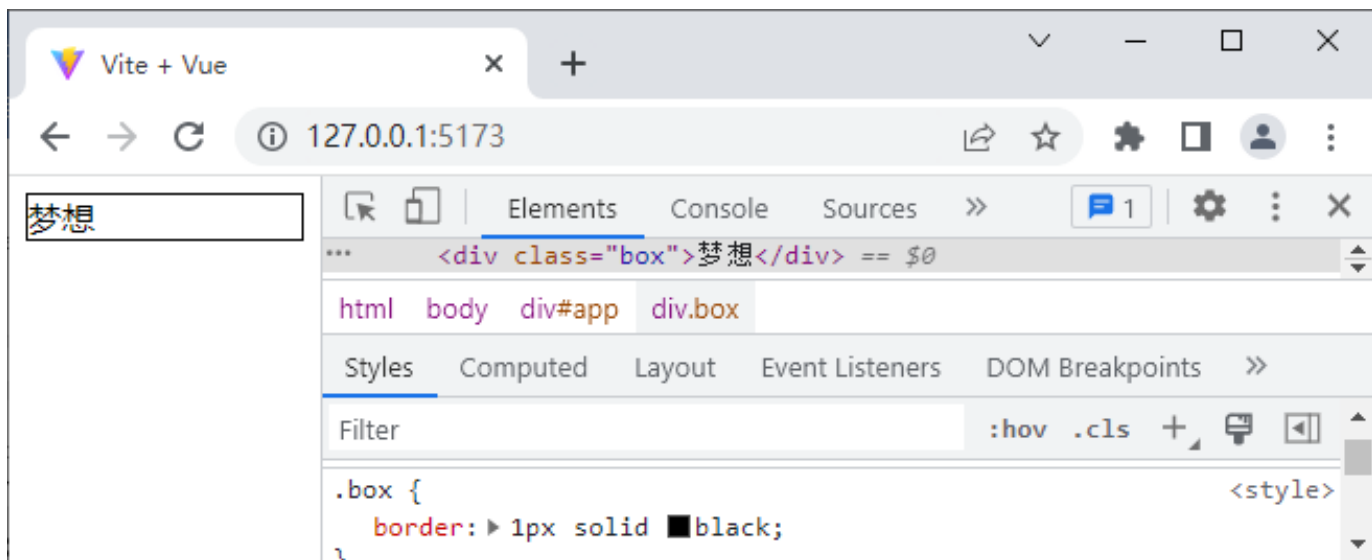
```
1  <template>
2    <div v-bind:class="className">梦想</div>
3
4  </template>
5  <script setup>
6    const className = 'box'
7  </script>
8
9  <style>
10    .box{
11      border: 1px solid ■■■black;
12    }
13  </style>
```

- 步骤2：修改src\main.js文件

```
JS main.js  ×  ▼ ClassStr.vue  ▼ Watch.vue
vue-demo > src > JS main.js
1  import { createApp } from 'vue'
2  import './style.css'
3  import App from './components/ClassStr.vue'
4
5  createApp(App).mount('#app')
```

»» 6.1 绑定class属性

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，为class属性绑定字符串的页面效果如下图所示。



» 6.1 绑定class属性

2. 将class属性值绑定为对象

- Vue中可以将class属性值绑定为对象，从而动态地改变class属性值。
- 对象中包含的属性名表示类名，属性值为布尔类型，属性值为true类名生效，否则类名不生效。示例如下。

```
<template>
  <div v-bind:class="{ className: isActive }"> </div>
</template>
<script setup>
import { ref } from 'vue'
const isActive = ref(true)
</script>
```

div元素的渲染后结果

```
<div class="className"> </div>
```

» 6.1 绑定class属性

- 使用v-bind绑定的class属性可以与普通的class属性共存，例如

```
<div class="className1" v-bind:class="{ className2:
isActive }"> </div>
```

在上述代码中，当isActive为true时，div元素的渲染结果如下。

```
<div class="className1 className2"> </div>
```

» 6.1 绑定class属性

- 在使用v-bind绑定class属性时，如果不想将对象类型的class属性值直接写在模板中，可以将属性值定义成一个响应式对象或一个返回对象的计算属性，示例代码如下。

```
<template>  
  <div v-bind:class="classObject1"> </div>  
  <div v-bind:class="classObject2"> </div>  
</template>
```

响应式对象

```
<script setup>  
import { reactive, ref, computed } from 'vue'  
const classObject1 = reactive({ className1: true })  
const isActive = ref(true)  
const classObject2 = computed(() => ({  
  className2: isActive.value  
}))  
</script>
```

计算属性

»» 6.1 绑定class属性

——为class属性绑定对象

■ 步骤1：创建src\components\ClassObject.vue文件

```
1 <template>
2   <div class="text" v-bind:class="{ box: isBox, border: isBorder }">
3     <div v-bind:class="{ inner: isInner }">春夏</div>
4     <div v-bind:class="classObject">秋冬</div>
5     <div v-bind:class="classMeal">三餐四季~</div>
6   </div>
7 </template>
8
9 <script setup>
10  import { ref, reactive, computed } from 'vue'
11  const isBox = ref(true)
12  const isBorder = ref(true)
13  const isInner = ref(true)
14  const isMeal = ref(true)
15  const classObject = reactive({ inner: true })
16  const classMeal = computed(() => ({
17    meal: isMeal.value
18  }))
19 </script>
```

对象

响应式对象

计算属性

» 6.1 绑定class属性

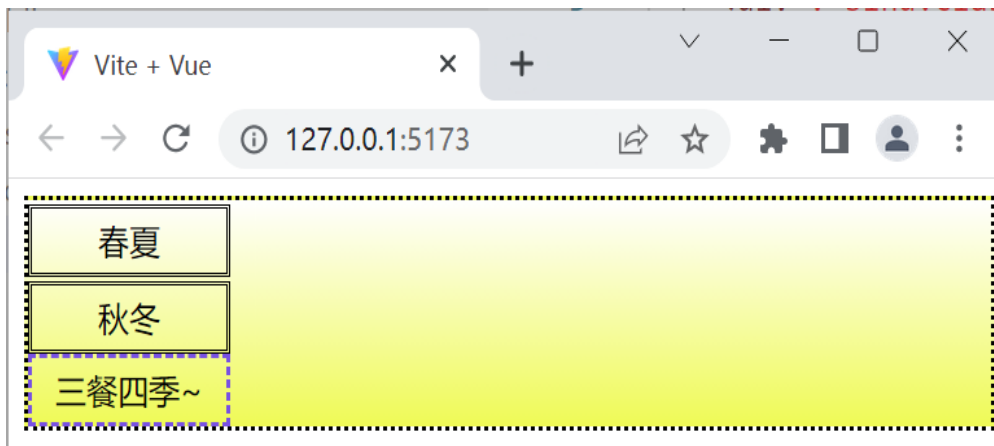
——为class属性绑定对象

■ 步骤2：在ClassObject.vue文件中定义样式

```
21 <style>
22 .text {
23   text-align: center;
24   line-height: 30px;
25 }
26
27 .box {
28   width: 100%;
29   background: linear-gradient(white, rgb(239, 250, 86));
30 }
31
32 .border {
33   border: 2px dotted black;
34 }
35
36 .inner {
37   margin-top: 2px;
38   width: 100px;
39   height: 30px;
40   border: 2px double black;
41 }
42
43 .meal {
44   width: 100px;
45   height: 30px;
46   border: 2px dashed rgb(120, 81, 227);
47 }
48 </style>
```

■ 步骤3：修改src\main.js文件。

```
JS main.js  X  ▼ ToRefs.vue  ▼ ClassObject.vue  ▼ ClassStr.v
vue-demo > src > JS main.js
1  import { createApp } from 'vue'
2  import './style.css'
3  import App from './components/ClassObject.vue'
4
5  createApp(App).mount('#app')
```



» 6.1 绑定class属性

3. 将class属性值绑定为数组

在Vue中，v-bind指令中的class属性值除了字符串和对象外，还可以是一个数组，用以动态地切换HTML的class属性，数组中的每个元素为要绑定的类名。示例如下

```
<template>
  <div v-bind:class="[className1, className2]"> </div>
</template>

<script setup>
import { ref } from 'vue'
const className1 = ref('active')
const className2 = ref('error')
</script>
```

div元素渲染结果

```
<div class="active error"> </div>
```

» 6.1 绑定class属性

- 有条件地切换列表中的class，可以使用三元表达式，例如

```
<div v-bind:class="[isActive ? className1 : className2]"> </div>
```

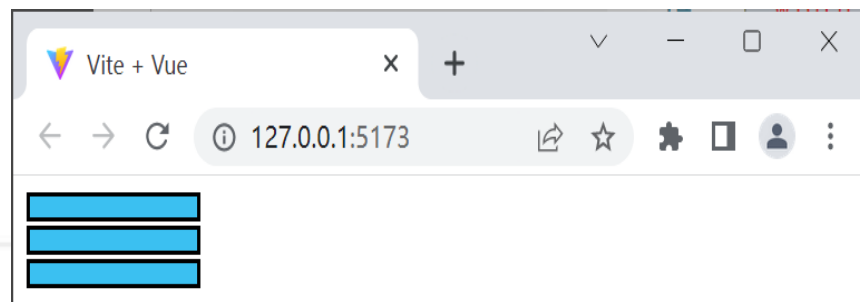
- class有多个条件时，在数组语法中也可以使用对象语法，例如

```
<div v-bind:class="[ { active: isActive }, className2]"> </div>
```

示例——将class属性值绑定为数组

```
>>> 1 <template>
2   <div v-bind:class="[activeClass,borderClass]"></div>
3   <div v-bind:class="[isActive?activeClass:'',borderClass]"></div>
4   <div v-bind:class="[active:isActive],borderClass]"></div>
5
6
7 </template>
8
9 <script setup>
10 import {ref} from 'vue'
11 const isActive = ref(true)
12 const activeClass = ref('active')
13 const borderClass = ref('border')
14 </script>
15 <style>
16 .active{
17   width: 100px;
18   height: 10px;
19   margin-bottom: 2px;
20   background-color: #39c0c0;
21 }
22 .border{
23   border: 2px solid black;
24 }
25 </style>
```

- 1 : 创建.vue文件
- 2 : 定义样式
- 3 : 修改main.js



»» 6.1 绑定class属性

写法：**:class= "xxx"**，**xxx**可以是字符串，对象，数组

Class绑定写法	适用场景
字符串	类名不确定，需动态获取。
对象	需要绑定多个样式，个数不确定，名称也不确定
数组	需要绑定多个样式，个数确定，名称确定，不确定用不用

»» 6.2 绑定style属性

- 在Vue中，通过更改元素的style属性也可以更改元素的样式，对style属性的操作也可以通过v-bind来实现。
- 使用v-bind绑定style属性时，可以将style属性值绑定为对象或数组。

» 6.2 绑定style属性

1. 将style属性值绑定为对象

- Vue中将style属性值绑定为对象时，该对象中的属性名表示CSS属性名，属性值为CSS属性值。示例如下。

```
<template>
  <div v-bind:style="{ color: 'red', 'font-size': '30px' }"> </div>
  <div v-bind:style="{ color: activeColor, fontSize: fontSize + 'px' }"> </div>
</template>

<script setup>
import { ref } from 'vue'
const activeColor = ref('red')
const fontSize = ref(30)
</script>
```

» 6.2 绑定style属性

- 除了将对象直接定义在模板中，还可以将对象定义在<script>标签中，示例如下。

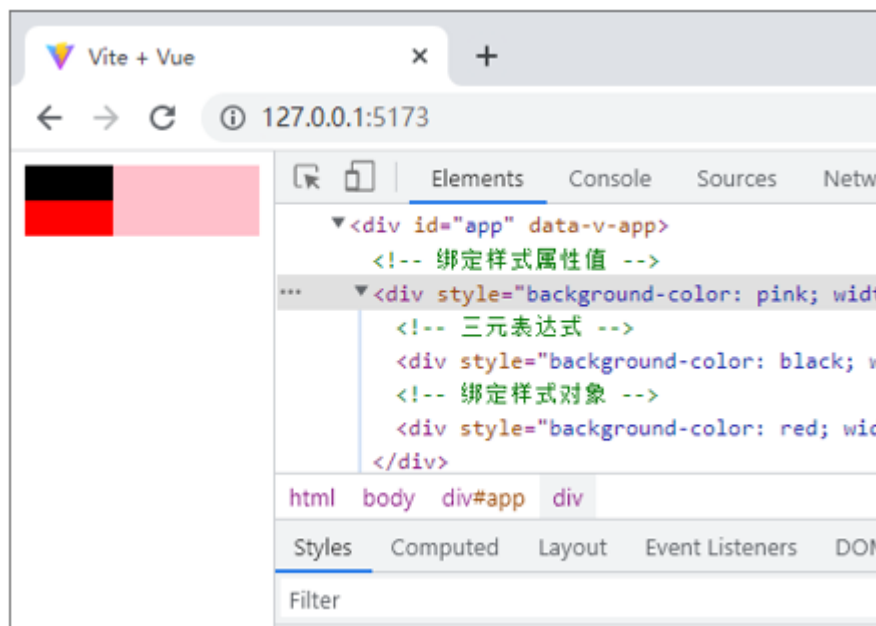
```
<template>
  <div v-bind:style="styleObject"> </div>
</template>

<script setup>
import { reactive } from 'vue';
const styleObject = reactive({
  color: 'red',
  fontSize: '12px'
})
</script>
```

»» 6.2 绑定style属性

——为style属性绑对象

```
1 <template>
2   <!-- 绑定样式属性值 -->
3   <div v-bind:style="{ 'background-color': pink, width: width, height: height + 'px' }">
4     <!-- 三元表达式 -->
5     <div v-bind:style="{ backgroundColor: isBlue ? blue : 'black', width: '50px', height: '20px' }"></div>
6     <!-- 绑定样式对象 -->
7     <div v-bind:style="myDiv"></div>
8   </div>
9 </template>
10
11 <script setup>
12 import { ref, reactive } from 'vue'
13 const isBlue = ref(false)
14 const blue = ref('blue')
15 const pink = ref('pink')
16 const width = ref('100%')
17 const height = ref(40)
18 const myDiv = reactive({
19   backgroundColor: 'red',
20   width: '50px',
21   height: '20px'
22 })
23 </script>
```



» 6.2 绑定style属性

2. 将style属性绑定为数组

将style属性绑定为数组，可以实现将多个样式应用到同一个元素上，示例代码如下。

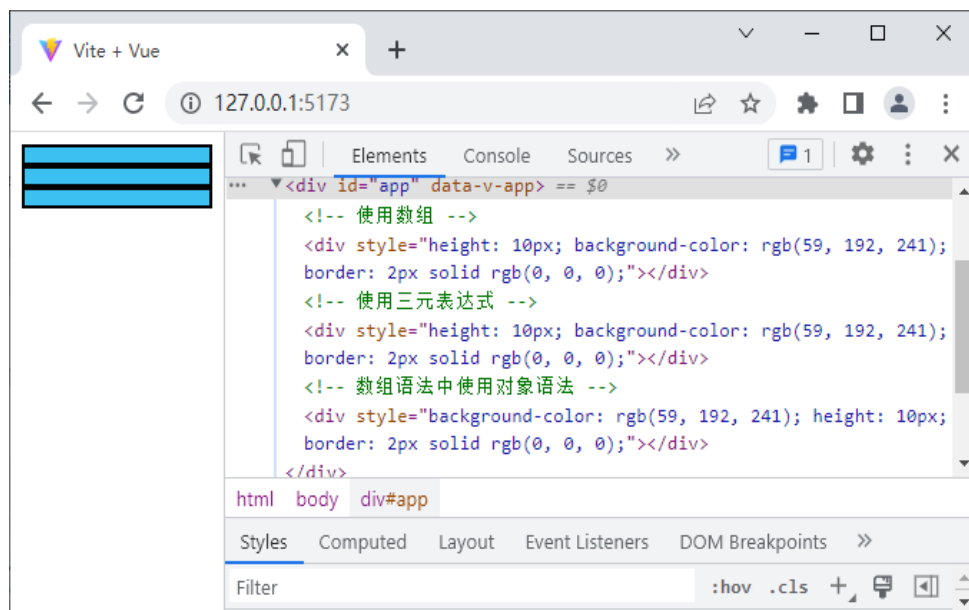
```
<div v-bind:style="[classObj1, classObj2]"> </div>
```

在上述代码中，数组中的每一个元素都是一个对象，例如classObj1和classObj2，对象中包含CSS属性名和属性值。每个对象需要在<script>标签中定义。

» 6.2 绑定style属性

——为style属性绑定数组

```
1 <template>
2   <!-- 使用数组 -->
3   <div v-bind:style="[activeClass, borderClass]"></div>
4   <!-- 使用三元表达式 -->
5   <div v-bind:style="[isActive ? activeClass : '', borderClass]"></div>
6   <!-- 数组语法中使用对象语法 -->
7   <div v-bind:style="[ { backgroundColor: 'rgb(59, 192, 241)', height: '10px' }, borderClass]"></div>
8 </template>
9
10 <script setup>
11 import { ref, reactive } from 'vue'
12 const isActive = ref(true)
13 const activeClass = reactive({
14   height: '10px',
15   backgroundColor: 'rgb(59, 192, 241)'
16 })
17 const borderClass = reactive({
18   border: '2px solid rgb(0, 0, 0)'
19 })
20 </script>
```



»» 示例

```
1  <script setup>
2  import {reactive,toRefs} from 'vue'
3  const state = reactive({
4    div1: { selected: false, active: false },
5    div2: { selected: true, active: false },
6    div3: { selected: false, active: true },
7    div4: { selected: true, active: true }
8  });
9  const { div1, div2, div3, div4 } = toRefs(state);
10 </script>
11
12 <template>
13   <div v-bind:class="div1">1</div>
14   <div v-bind:class="div2">2</div>
15   <div v-bind:class="div3">3</div>
16   <div v-bind:class="div4">4</div>
17
18
19 </template>
```

```
21 <style>
22   div>div {
23     width: 60px;
24     height: 60px;
25     border: 2px solid #bbb;
26     text-align: center;
27     line-height: 60px;
28     font-size: 30px;
29     margin: 10px;
30     float: left;
31   }
32   .selected {
33     border-color: #000;
34   }
35   .active {
36     background: #bbb;
37   }
38 </style>
```

1

2

3

4